8 Constant Reloading

When `config.cache_classes` is false Rails is able to reload autoloaded constants.
For example, in you're in a console session and edit some file behind the scenes, the code can be reloaded with the `reload!` command:

```
> reload!
```

When the application runs, code is reloaded when something relevant to this logic changes. In order to do that, Rails monitors a number of things:

- `config/routes.rb`.
- Locales.

- Ruby files under `autoload_paths`.
- `db/schema.rb` and `db/structure.sql`.

If anything in there changes, there is a middleware that detects it and reloads the code.

Autoloading keeps track of autoloaded constants. Reloading is implemented by removing them all from their respective classes and modules using `Module#remove_const`. That way, when the code goes on, those constants are going to be unknown again, and files reloaded on demand.
This is an all-or-nothing operation, Rails does not attempt to reload only what changed since dependencies between classes makes that really tricky. Instead, everything is wiped.

9 Module#autoload isn't Involved

`Module#autoload` provides a lazy way to load constants that is fully integrated with the Ruby constant lookup algorithms, dynamic constant API, etc. It is quite transparent.
Rails internals make extensive use of it to defer as much work as possible from the boot process. But constant autoloading in Rails is **not** implemented with `Module#autoload`.
One possible implementation based on `Module#autoload` would be to walk the application tree and issue `autoload` calls that map existing file names to their conventional constant name.
There are a number of reasons that prevent Rails from using that implementation.

For example, `Module#autoload` is only capable of loading files using `require`, so reloading would not be possible. Not only that, it uses an internal `require` which is not `Kernel#require`.
Then, it provides no way to remove declarations in case a file is deleted. If a constant gets removed with `Module#remove_const` its `autoload` is not triggered again. Also, it doesn't support qualified names, so files with namespaces should be interpreted during the walk tree to install their own `autoload` calls, but those files could have constant references not yet configured.
An implementation based on `Module#autoload` would be awesome but, as you see, at least as of today it is not possible. Constant autoloading in Rails is implemented with `Module#const_missing`, and that's why it has its own contract, documented in this guide.

10 Common Gotchas

10.1 Nesting and Qualified Constants

Let's consider

```
module Admin
```